Proceedings of Machine Learning Research **318** (2026)
The 39th Canadian Conference on Artificial Intelligence

Canadian AI 2026
25–29 May 2026, Vancouver, BC

VOLTS: Validated Output through Logit Tree Search for Reliable PDDL Planning with Small Language Models

First Author[†,*], Second Author[‡], Third Author[◇]
[†] Affiliation One
[‡] Affiliation Two
[◇] Affiliation Three

Abstract

Autonomous agents that must run on edge hardware cannot afford the compute footprint of frontier LLMs, yet they still need dependable task-planning. We address this gap by showing how a single pass with Llama 3.1 8B, 4-bit Small Language Model (SLM) can generate syntactically correct plans in the symbolic-planning formalism Planning Domain Definition Language (PDDL) while respecting tight memory and latency budgets. VOLTS rests on three ideas. **(1) Action-token fine-tuning:** the SLM is fine-tuned on a custom vocabulary where every token encodes a complete grounded action, giving the model strong task heuristics without expanding its size. **(2) Real-time validator:** a lightweight symbolic module checks each candidate token against the current state during decoding, guaranteeing that any plan emitted contains no hallucinated or infeasible actions. **(3) Parallel branching search:** when several validated actions appear promising, VOLTS explores them in parallel branches within the same forward pass, preserving single-pass efficiency while widening search. Evaluated on 2 000 problems (500 each in the IPC Blocksworld, Logistics, DriverLog, and Rover domains), VOLTS returns valid plans for 76% of tasks. Those plans average 1.08× the length of solutions from the classical Fast Downward planner, far outperforming GPT-4o (7% validity) and a fine-tuned baseline without in-loop validation (0.13%). Unlike Tree-Planner or LLM-Modulo frameworks, VOLTS validates per token inside a single inference pass, eliminating costly iterative cycles. By coupling resource-aware neural guidance with deterministic symbolic checks, VOLTS opens the door to reliable on-device planning for robots, drones, and embedded IoT agents where every millisecond and megabyte counts.

Keywords: Automated Planning, PDDL, Small Language Models, Neuro-Symbolic AI, Tree Search, Validation

1. Introduction

The remarkable advancements in Large Language Models (LLMs) have spurred intense interest in their application to complex cognitive tasks, prominently including automated planning. Automated planning, often formalized using symbolic planning standards such as the Planning Domain Definition Language (PDDL) [1], seeks sequences of valid actions to transform an initial world state into a desired goal state. Symbolic planning casts decision-making as a search over explicit state-transition models, deriving a sequence of actions that provably moves the system from an initial state to a goal. PDDL provides a concise, declarative syntax for specifying each action's preconditions and effects, which helps overcome the inherent ambiguity of natural language, facilitating the precise state transition modeling necessary for robust planning [2]. One might argue that LLMs possess vast world knowledge and emergent reasoning capabilities [3, 4], yet harnessing them as autonomous, reliable planners within such formal frameworks, especially when aiming for both correctness and efficiency with Small Language Models (SLMs), remains a significant challenge [5, 6].

A growing consensus, supported by numerous studies [5–7], indicates that LLMs, even frontier models, struggle with core planning requirements like strict constraint adherence, verifiable correctness, implicit state tracking over long horizons, and reliable multi-step

[*] corresponding_author@example.ca

This article is © 2026 by author(s) as listed above. The article is licensed under a Creative Commons Attribution (CC BY 4.0) International license (https://creativecommons.org/licenses/by/4.0/legalcode), except where otherwise indicated with respect to particular material included in the article. The article should be attributed to the author(s) identified above.

reasoning [8]. They are prone to hallucination [9], may generate infeasible or irrelevant actions [10], lack inherent mechanisms for self-verification [5], and their performance on planning benchmarks remains limited. Consequently, research increasingly focuses on integrating LLMs within broader, often hybrid, frameworks (LLM-Modulo approaches) [5] or employing structured reasoning paradigms. Prominent methods like ReAct [11] interleave reasoning and acting with environmental feedback, while tree-based search strategies like Tree of Thoughts (ToT) [12] or Tree-Planner [13] explicitly explore multiple reasoning paths. However, these iterative approaches can be inefficient, requiring numerous interactions or passes, and crucially, often lack mechanisms to guarantee plan validity during the generation process itself.

This paper introduces VOLTS (**V**alidated **O**utput through **L**ogit **T**ree **S**earch), a novel framework that enables a fine-tuned, 4-bit quantized SLM (Llama 3.1 8B with Parameter-Efficient Fine-Tuning) to generate PDDL plans with proven validity and high efficiency. Figure 1 illustrates how VOLTS explores multiple promising paths in parallel while guaranteeing that every retained partial plan remains symbolically valid, thereby eliminating hallucinations without sacrificing search breadth. VOLTS distinguishes itself by uniquely combining (1) fine-tuning of SLMs specifically for PDDL planning, (2) a custom vocabulary aligning tokens with PDDL actions/parameters, enabling (3) real-time, token-level PDDL validation during inference, integrated within (4) a logit-based tree search.

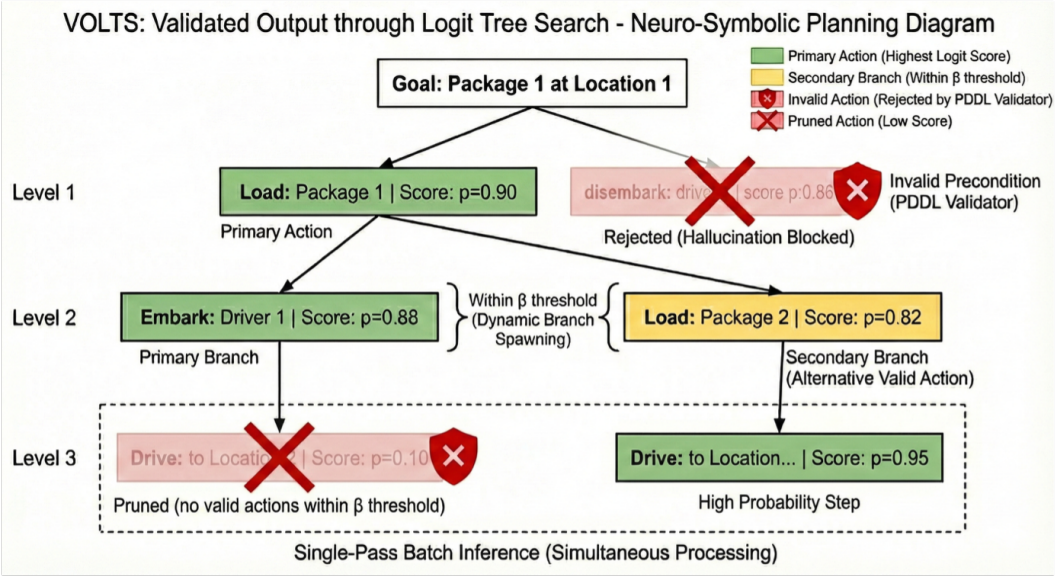

Figure 1. VOLTS: Validated Output through Logit Tree Search – Neuro-Symbolic Planning Diagram. The figure illustrates how VOLTS explores multiple promising paths in parallel while guaranteeing that every retained partial plan remains symbolically valid, thereby eliminating hallucinations without sacrificing search breadth. Green nodes represent primary actions (highest logit score), yellow nodes represent secondary branches (within β threshold), red-crossed nodes indicate invalid actions rejected by the PDDL validator, and pruned nodes show low-score actions eliminated from consideration.

This synergy addresses key limitations of prior work: it guarantees plan validity unlike pure LLM approaches, offers a deterministic and formal grounding unlike some affordance-based methods, achieves efficiency and single-pass generation unlike iterative or post-hoc validation techniques, and specifically makes reliable PDDL planning feasible on computationally constrained SLMs. Importantly, VOLTS does not aim to replace classical planners

in domains they already handle well. Rather, it targets two complementary gaps that symbolic methods leave open: *(i)* classical heuristic search must ground and evaluate an action space that grows combinatorially with domain objects, whereas a fine-tuned SLM compresses this space into a learned distribution that proposes only a handful of probable next actions, sidestepping the grounding bottleneck; and *(ii)* pre-trained language models encode cross-domain world knowledge that may eventually enable generalization to novel tasks with limited retraining. This form of transfer is absent from classical planning.

Our central hypothesis is that fine-tuning equips SLMs with effective, albeit imperfect, planning heuristics discernible in their output logits, leveraging the LLM's predictions similarly to heuristic guides [14]. VOLTS's inference strategy manages a set of active plan branches. In each step, the SLM proposes potential next actions for all branches simultaneously. These candidates are immediately evaluated by the validator, which operates on a custom vocabulary treating PDDL actions and parameters as individual tokens, ensuring that only symbolically valid actions extend the search tree. Inspired by structured exploration principles [12], and to foster exploration of diverse yet promising avenues, new branches are dynamically spawned from validated alternatives if their logit scores are within a β-ratio of the leading choice for a given branch, subject to a limit of $k_{\max}$ concurrent branches. This method ensures that all explored plan segments are valid by construction and efficiently leverage the SLM's predictive capabilities across multiple hypotheses. Our empirical evaluation, demonstrating an average planning validity of 76.4% across standard benchmarks, strongly supports this integrated approach.

The main contributions of this work are:

(1) A novel planning framework enabling fine-tuned SLMs on custom domain vocabulary to generate syntactically valid PDDL plans that reach the goal state through a parallel, validated, logit-guided tree search.
(2) A custom PDDL grounding module for STRIPS [15] domains built in Python, designed to validate actions against the current state.
(3) Empirical demonstration, via comparisons with GPT-4o/GPT-o3-mini and ablated baselines on fine-tuned Llama 3.1 8B, that our VOLTS approach achieves high planning success, high efficiency at low costs with an SLM across standard PDDL domains (Blocksworld, Logistics, Driverlog, Rover).

2. Related Work

The challenge of leveraging LLMs for automated planning, particularly within formalisms like PDDL, has attracted significant research attention. Existing approaches can be broadly categorized based on how they utilize LLMs and address the inherent limitations of these models in planning tasks.

2.1. LLMs as Zero-Shot or Few-Shot Planners

Initial explorations treated LLMs as direct, off-the-shelf planners, prompting them with task descriptions and expecting executable plans as output [7]. While demonstrating some capacity for generating plausible steps, these methods often suffer from severe limitations regarding plan validity, correctness, and adherence to domain constraints [5, 6]. LLMs frequently hallucinate actions, misunderstand preconditions/effects, and struggle with long-horizon consistency [8]. Simple fine-tuning on planning data, while potentially improving domain adaptation, has often proven insufficient on its own to guarantee reliable plan generation, especially for complex domains or smaller models.

2.2. **Grounding LLM Planners**

Recognizing the lack of physical and environmental grounding in LLMs, several works focus on connecting LLM outputs to agent capabilities and world state. **SayCan** [16] proposes combining LLM likelihood scores for skill usefulness with learned affordance functions (value functions) representing skill feasibility. **ProgPrompt** [17] grounds LLMs by providing programmatic specifications of available actions and objects within the prompt itself, using assertions for limited state feedback. Other approaches focus on grounding semantic skills across different domains [18] or incorporating memory systems for better state awareness [19]. While improving situatedness (i.e., ensuring action choices reflect the current situation), these methods often rely on probabilistic grounding which exposes the plan to hallucinations and shows degrading performance in small parameter models (SayCan), carefully engineered prompts require significant domain knowledge and are akin to designing a symbolic solver to solve the problem (ProgPrompt), complex memory architectures, or do not provide formal guarantees of plan validity according to strict symbolic rules like PDDL. Our work differentiates itself by tightly integrating a custom PDDL validator during the inference process where candidate actions are grounded at each inference step.

2.3. **Structured Reasoning and Search with LLMs**

Structured reasoning frameworks (e.g., CoT, ToT/GoT, ReAct) and planning-oriented tree search methods (e.g., Tree-Planner/Vote-Tree-Planner, MCTS, A*) improve exploration but typically require multiple model calls and/or validate only after generating larger chunks of text [2, 11, 12, 20–22]. In contrast, VOLTS validates each candidate action token during decoding and uses logits to guide parallel branching in a single pass.

2.4. **LLMs and PDDL**

The interface between LLMs and PDDL is explored from several angles. Some works investigate using LLMs to generate PDDL domain models or problem files from natural language descriptions. Others focus on automating the generation of natural language prompts from PDDL specifications to query LLMs [23]. A distinct line of work uses LLMs to synthesize generalized planning programs (e.g., in Python) capable of solving PDDL tasks [24]. These contrast with our objective, which is to use LLMs to generate the PDDL plan itself within a given domain, while guaranteeing its validity. Hybrid approaches explicitly combine LLM reasoning (e.g., for goal selection or heuristic guidance) with classical PDDL planners that handle the actual plan generation. Our method differs by using the LLM for plan generation but constraining it tightly via the integrated validator. In practice, this supports one-shot plan synthesis by constraining generation to valid continuations, avoiding iterative next-action selection across a combinatorial action space that quickly increases noise and degrades action selection reliability (context dilution).

2.5. **LLM-Modulo Framework**

Recognizing the consensus that LLMs alone are insufficient for reliable planning, the LLM-Modulo framework [5] explicitly advocates combining LLMs (as approximate knowledge sources or candidate generators) with external, model-based symbolic verifiers or solvers. This paradigm acknowledges the strengths and weaknesses of both neural and symbolic approaches. Our work aligns strongly with this perspective but implements a particularly tight and efficient integration. By embedding a fast PDDL validator directly into the SLM's token generation loop via a custom vocabulary and logit-based search, we

achieve the validation benefits of an external module without the latency of typical iterative request-validate-critique cycles, enabling guaranteed validity in a single pass even with SLMs.

3. Methodology

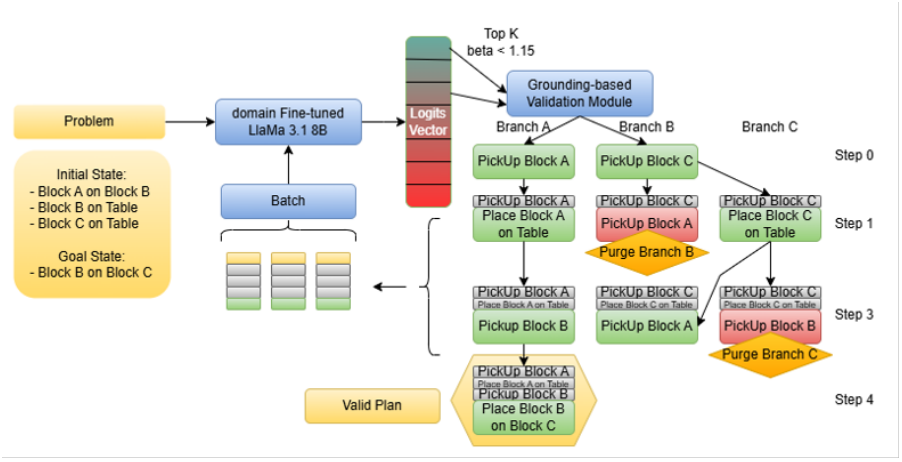

Figure 2. VOLTS Algorithm with fine-tuned model and validation module. This approach allows a logit-tree exploration by predicting valid next actions a_{valid}, grounded at each step t with the current state s_t of each branch B by the Validation Module. The Top-K actions with a $\beta < 1.15$ are selected from the candidate actions a_{cand} generated by the domain fine-tuned Language Model. The branches are dynamically managed to only keep $\{a_{\text{valid}}\}$ sequences. The resulting branches are batched and fed back into the Language Model to generate the next logit vector l_i for each branch B.

Our approach, VOLTS, enables reliable and efficient PDDL task planning using a Small Language Model (SLM), specifically, a 4-bit quantized Llama 3.1 8B, by overcoming standard fine-tuning limitations and iterative validation inefficiencies. The core idea involves combining fine-tuning of SLM specifically for PDDL planning on a custom vocabulary aligning tokens with domain actions/parameters, enabling real-time, token-level PDDL validation during inference (see Figure 2), integrated within a logit-based tree search. The fine-tuning approach utilizes parameter-efficient fine-tuning with Low Rank Adaptation (LoRA) with Principal Singular Values and Singular Vector Decomposition (PiSSA) as initialization [25, 26], reducing computational and memory requirements when retraining the embeddings with the custom domain vocabulary. The validation is done with a fast symbolic validator coded in Python. The final algorithm integrates all these elements in a batched parallel inference strategy based on logit exploration and dynamic branching as seen in Figure 2. This synergy generates guaranteed valid PDDL plans while maintaining computational efficiency.

During fine-tuning, only the LoRA adapter weights (initialized via PiSSA) and specific vocabulary-related layers are updated, while the 4-bit quantized base model weights remain frozen. The fine-tuning objective follows the standard autoregressive language modeling goal: predicting the next token in valid PDDL plan sequences corresponding to given PDDL problems, using datasets derived from Blocksworld, Logistics, and Driverlog (∼20,000 examples each).

3.1. **Custom Vocabulary Integration**

A critical bottleneck for real-time validation is standard sub-word tokenization, which fragments PDDL actions and parameters. To address this, we construct a custom vocabulary by extracting all unique action definitions and relevant parameter instances from the target PDDL domains. Each distinct action and parameter are assigned a unique token ID as seen in Figure 3.

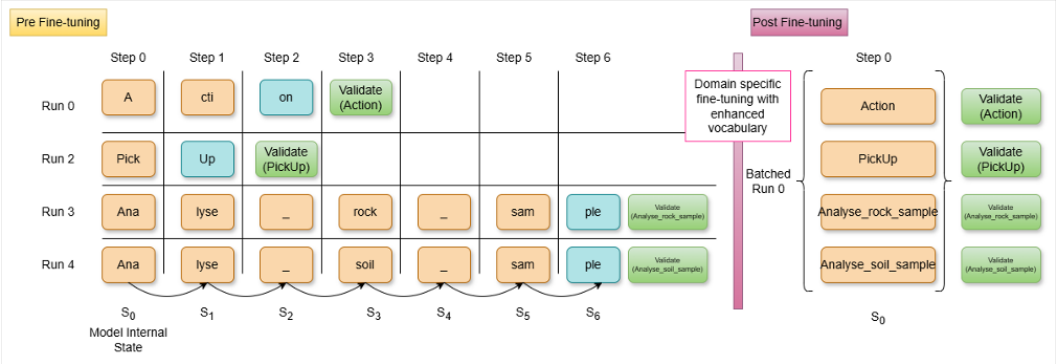

Figure 3. Domain vocabulary fine-tuning comparison. **Left (Pre Fine-tuning):** Standard sub-word tokenization fragments PDDL actions across multiple tokens (e.g., "Pick", "Up" requiring 2 steps, or "Ana", "lyse", "_", "rock", "_", "sam", "ple" requiring 7 steps), making validation impossible until the complete action is assembled. **Right (Post Fine-tuning):** With domain-specific vocabulary, each complete PDDL action (e.g., `PickUp`, `Analyse_rock_sample`) is a single token, enabling immediate validation at each generation step. This allows batched parallel processing of multiple candidate actions in a single forward pass.

Integrating this custom vocabulary requires modifications to the model architecture. Specifically, the model's input embedding layer and final output layer (language modeling head) are resized to accommodate the new vocabulary size. These embedding and output layers are trained alongside the LoRA adapters during the PEFT process, allowing the model to learn representations for the custom action/parameter tokens and predict logits over this new vocabulary space. The training data (plans) are re-tokenized using this custom vocabulary. This process necessitates sufficient training data (20k–50k examples per domain) for the model to learn effectively within the expanded vocabulary and adapter space.

3.2. **Action Grounding Module**

Classical planning relies on symbolic validators to check action preconditions and effects. While various PDDL validators exist, they are generally too slow for integration within an LLM's token generation loop requiring either the use of subprocesses, complete plans or use libraries with large overheads not designed for rapid action grounding. Therefore, we developed a custom PDDL action validator implemented in Python. This validator dynamically assesses the validity of a candidate action token (from the custom vocabulary) at each generation step. Given the current symbolic state (implicitly maintained and updated) and a candidate action token, the validator efficiently checks if the corresponding PDDL action's preconditions are satisfied. It returns a Boolean indicating validity. The speed of this validator is paramount for the real-time integration within the logit-based tree search.

3.3. **Logit-Based Tree Search Inference**

Standard autoregressive generation, which greedily selects the highest-probability action token, often falters in complex planning tasks as this choice may not be valid or part of an optimal path. Similarly, methods involving iterative validation of complete, independently generated plans can be highly inefficient due to repeated querying of LLMs. VOLTS addresses these shortcomings by conducting a parallel, logit-guided tree search where multiple potential plan sequences (branches) are explored simultaneously. Each proposed action within these branches is validated in real-time against PDDL semantics before being added to the plan, ensuring constructive correctness while attempting to balance the exploration vs exploitation through the β branching factor and $k_{\max}$ branches.

3.4. **VOLTS Algorithm**

The core of the VOLTS inference strategy lies in managing a set $\mathcal{B}$ of active branches, where each branch (P_i, s_i) consists of a partial plan P_i and its corresponding current symbolic state s_i. The search proceeds in steps, maintaining a bounded number of active branches to manage computational resources.

(1) **Batch Candidate Generation:** A batch input $\mathcal{I}$ is constructed from the current states and partial plans of all active branches currently in $\mathcal{B}$. The fine-tuned SLM M_θ processes this batch $\mathcal{B}$ in a single forward pass, yielding a set of logit vectors $\mathcal{L}$, where each $l_i \in \mathcal{L}$ corresponds to an active branch $(P_i, s_i) \in \mathcal{B}$.

(2) **Per-Branch Validation and Extension:** For each branch (P_i, s_i) and its associated logits l_i:
   - (a) Candidate next action tokens are derived from l_i.
   - (b) The Real-Time PDDL Validator V evaluates these candidates against the current state s_i. This yields a set A_{valid} containing only valid PDDL actions for the current state along with their corresponding logit scores.
   - (c) If A_{valid} is empty for the current branch, it indicates that no valid immediate action was found among the candidates, and this branch is pruned.
   - (d) If A_{valid} is not empty, the action a_{primary} with the highest logit score within A_{valid} is selected as the primary candidate to extend the current branch. The plan P_i is updated to $P_i' = P_i + [a_{\text{primary}}]$, and the state s_i transitions to $s_i' = \text{ApplyAction}(s_i, a_{\text{primary}})$. If this new state s_i' satisfies the goal conditions G, the plan P_i' is returned as a solution.

(3) **Dynamic Branch Spawning Exploration:** To explore promising alternative paths, VOLTS incorporates a dynamic branching mechanism. For every other valid action $a_{\text{other}} \in A_{\text{valid}}$ (where $a_{\text{other}} \neq a_{\text{primary}}$), its score is compared to that of a_{primary}. If the ratio $(\text{score}(a_{\text{primary}})/\text{score}(a_{\text{other}}))$ is less than a predefined branching factor β (e.g., 1.15), it indicates that a_{other} is a sufficiently promising alternative. Provided that the current number of branches is less than $k_{\max}$, a new branch is created from (P_i, s_i) applying action a_{other}.

(4) **Step Update:** After processing all branches in the current set $\mathcal{B}$, the set of active branches for the next step is updated to $\mathcal{B} = \mathcal{B}_{\text{next}} \cup \mathcal{B}_{\text{new}}$. The step t is incremented.

(5) **Termination:** The loop continues until a plan is returned, $\mathcal{B}$ becomes empty (no more valid paths to explore), or the maximum plan length/time $T_{\max}$ is reached. If no solution is found, Failure is returned.

This batched, parallel processing, combined with dynamic branching controlled by β and $k_{\max}$, allows VOLTS to efficiently explore a diverse set of valid plan trajectories. The integrated real-time validation ensures that all explored paths strictly adhere to PDDL semantics by construction.

4. Experimental Setup

This section details the experimental design employed to rigorously evaluate our proposed VOLTS framework. We benchmark VOLTS against several baseline methods across standard PDDL domains, focusing on plan validity, plan quality, and the impact of our core algorithmic contributions.

4.1. Datasets and Domains

We evaluate our method on four diverse PDDL benchmark domains sourced from PlanBench [6] and the International Planning Competition (IPC) STRIPS track: Blocksworld, Logistics, DriverLog, and Rover. These domains present a variety of planning challenges, including state-space size and action combinatorics. For fine-tuning VOLTS and the ablation baseline, we generated approximately 20,000 unique problem-plan pairs for each domain using classical planners. For evaluation, distinct sets of problems were used; for example, the DriverLog evaluation comprised 500 instances with varying entity counts (locations: 2–4, drivers: 2–4, packages: 2–5, trucks: 2–6).

4.2. Models and Baselines

Our comparative analysis includes the following:

- **Classical Planner (Baseline):** We use Fast Downward [27] to generate plans, providing a ground truth for plan length (ratio 1.0) and a 100% validity reference.
- **GPT-4o:** OpenAI's frontier model, prompted in a zero-shot fashion with PDDL domain and problem files.
- **GPT-o3-mini*:** An OpenAI model noted for reasoning capabilities. *This model utilizes a reasoning approach akin to CoT [20] that it was specifically fine-tuned to produce, without external validation tools.
- **Vanilla Llama 3.1 8B:** The base Llama 3.1 8B model without any domain-specific fine-tuning, prompted directly.
- **Fine-Tuned Llama 3.1 8B (Ablation Baseline):** This crucial baseline uses the Llama 3.1 8B model, fine-tuned with the same PEFT techniques (4-bit quantization, LoRA, PiSSA [25, 26]) and custom PDDL vocabulary as VOLTS. However, it employs standard autoregressive decoding during inference, without the VOLTS integrated real-time validation, parallel branch management, or β-branching logic.
- **VOLTS (Ours):** Our proposed framework, utilizing the fine-tuned Llama 3.1 8B with the Logit-Based Tree Search Inference mechanism.

4.3. Implementation Details

Fine-Tuning: The Llama 3.1 8B model was quantized to 4-bits. PEFT used LoRA adapters initialized with PiSSA. Training used the custom PDDL vocabulary and the domain-specific datasets ($\sim$20,000 examples each).

VOLTS Inference Parameters: The branching factor, which is determined by setting a high starting branching factor (10) and reducing it until the perplexity reaches at most 3 [28], β was set to 1.15. Values above 1.45 lead to significant quality degradation with the fine-tuned model. The maximum number of concurrent active branches $k_{\max}$ was set to 4 due to hardware limitations. Within each branch, for deriving A_{cand} from logits l_i, an initial pool of the top unique tokens were considered before validation. The maximum plan length $T_{\max}$ was set appropriately for each domain (e.g., 50–100 steps).

Hardware: All fine-tuning and local model inference (Llama variants) were performed on a workstation equipped with an NVIDIA RTX 4090 GPU (24GB VRAM).

4.4. Evaluation Metrics

Performance is assessed using the following metrics:

- **Plan Validity (%):** The primary metric, defined as the percentage of generated plans that are syntactically and semantically valid according to the PDDL domain definition and successfully achieve the specified goal state from the initial state.
- **Plan Length (Ratio):** For all valid plans generated, this is the ratio of the generated plan length (number of actions) to the baseline plan length found by Fast Downward [27]. Ratios close to 1.0 indicate near-baseline plan length (lower is better).

5. Results

This section presents the empirical results of VOLTS compared against the established baselines across the four PDDL domains. The findings underscore VOLTS's capability to generate valid and near-baseline plans with a resource-efficient SLM.

Method	**Average Validity**	**Plan Length Ratio**
Fast Downward (Symbolic Baseline)	100%	1.00
GPT-4o	7.2%	**0.52**
GPT-o3-mini (reasoning)	48.9%	0.55
Vanilla Llama 3.1 8B	0.0%	–
Fine-tuned Llama 3.1 8B (Ablation)	0.13%	5.88
VOLTS (Ours)	**76.4%**	1.08

Table 1. Overall Performance Summary across all four PDDL domains.

5.1. Plan Validity

VOLTS demonstrates a striking improvement in plan validity. As summarized in Table 1, VOLTS achieves an average validity of 76.4% across all tested domains.

Superiority over Large Models: This performance significantly surpasses that of much larger models like GPT-4o, which averaged only 7.2% validity, and also outperforms the reasoning-focused GPT-o3-mini* (48.9% validity). This highlights the critical role of specialized mechanisms for enforcing formal constraints, which even advanced LLMs lack natively for PDDL.

Impact of VOLTS Search Strategy (Ablation): The most critical comparison is with the fine-tuned Llama 3.1 8B ablation baseline. Despite identical model size, fine-tuning data, PEFT methods, and custom vocabulary, this baseline (relying on standard autoregressive decoding) achieved a mere 0.13% average validity (e.g., Blocksworld: 0.02%, Logistics: 0.15%, DriverLog: 0.13%, Rover: 0.22%). This vast disparity underscores that fine-tuning alone is insufficient for reliable planning with SLMs; the integrated validation and guided parallel search of VOLTS are essential. The Vanilla Llama 3.1 8B achieved 0% validity, confirming that fine-tuning on planning data is a necessary preliminary step.

Domain-Specific Validity: VOLTS exhibited robust performance across diverse domains, achieving 64.6% on Blocksworld, 74.6% on Logistics, an impressive 83.6% on DriverLog, and 82.8% on Rover.

5.2. Plan Length Quality

Beyond validity, VOLTS generates plans of high quality, closely approaching baseline lengths.

Near-Optimal Plans: Valid plans from VOLTS average only 1.08× the length of Fast Downward solutions. Domain-specific ratios: Blocksworld (1.4×), Logistics (1.01×), DriverLog (1.12×), and Rover (0.77×).

Comparison with Baselines: The fine-tuned ablation baseline averaged 5.88× baseline length when valid (e.g., Blocksworld 9.7×, Logistics 3.7×). GPT-4o (0.52×) and GPT-o3-mini (0.55×) show shorter ratios, but these reflect their low validity rates solving only trivial problems.

These results collectively demonstrate that VOLTS empowers an SLM to achieve high success rates in generating valid, near-baseline PDDL plans, far exceeding both larger, general-purpose LLMs and standard fine-tuning approaches for this specialized task.

6. Conclusion

This paper addresses the significant challenge of enabling Small Language Models to perform reliable and efficient automated planning within formal frameworks like PDDL. Our proposed framework, VOLTS, demonstrates a step forward in achieving this goal, leveraging a fine-tuned SLM. We emphasize that VOLTS is not positioned as a replacement for classical planners in domains they handle well. Rather, it targets a 2 key gaps that symbolic methods leave open. *(i) Combinatorial scalability.* Classical heuristic search must ground and evaluate an action space that grows combinatorially with the number of domain objects. A fine-tuned SLM implicitly compresses this space into a learned distribution, proposing only a small set of probable next actions at each step and thereby avoiding the grounding bottleneck that can render symbolic planners intractable in large-scale domains. *(ii) Commonsense transfer.* Pre-trained language models encode world knowledge that extends well beyond any single planning domain. While this paper evaluates VOLTS on standard PDDL benchmarks, the broader motivation is that such cross-domain priors may eventually allow neural planners to generalize to novel tasks with limited additional training. This form of transfer has no direct analog in classical planning.

The empirical results clearly show VOLTS's capability, with the fine-tuned Llama 3.1 8B achieving an average plan validity of 76.4% and near-baseline plan lengths (1.08× baseline) across diverse PDDL domains. This performance, particularly when contrasted with the negligible success of the identically fine-tuned ablation baseline (0.13% validity), underscores a key contribution: the VOLTS architecture, with its integrated real-time PDDL validator and parallel logit-guided tree search, is crucial for reliable planning in SLMs. Fine-tuning alone, even with a domain-specific vocabulary, is insufficient.

A primary problem VOLTS tackles is the efficiency of generating valid plans, especially when compared to common approaches for enhancing reliability in larger foundation models. Many existing strategies rely on iterative feedback loops, multiple API calls to cloud-based models (e.g., for critique-and-revise cycles or complex multi-step reasoning like LLM-Modulo, ReAct or Graph-of-Thoughts [5, 11, 29]), which can incur significant latency, token costs, and computational overhead. VOLTS offers a more streamlined path. Its single-pass generation process, where a valid plan is constructed through one continuous execution flow, even if involving multiple internal branches, is inherently more efficient. The custom PDDL vocabulary facilitates instant token-level validation, allowing the system to prune invalid search paths immediately for all concurrently explored branches processed in a batch. This dynamic β-controlled branching explores promising alternatives without the need for costly full plan regeneration or separate external validation calls, making it highly suitable for resource-constrained SLMs designed for local deployment.

By leveraging the SLM's learned heuristics through its logits [14] while strictly enforcing symbolic correctness, VOLTS aligns with the LLM-Modulo paradigm [5] in a particularly tight and efficient manner. While performance varied across domains (e.g., 64.6%

in Blocksworld vs. 83.6% in DriverLog), this highlights areas for further refinement rather than detracting from the core architectural advantages.

6.1. **Future Work**

Despite its promising results, VOLTS has limitations that motivate several research directions. The reliance on substantial domain-specific training data (20k–50k examples) can be a bottleneck for new domains, calling for few-shot PEFT and cross-domain transfer techniques. The custom vocabulary's scalability across highly diverse PDDL specifications, the validator's efficiency with complex preconditions, and the limited candidate breadth per branch (top 5) all present open challenges. Future work will target explicit plan cost optimization, support for richer PDDL features (numeric fluents, temporal actions, conditional effects), adaptive branching strategies, and broader application of the core principles to other structured generation tasks on edge devices.

Future work includes improving plan optimality, supporting richer PDDL (e.g., numeric/temporal constructs), reducing data requirements via transfer/few-shot PEFT, and exploring adaptive branching/search strategies and broader applications.

References

[1] M. Fox and D. Long. "PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains". In: *Journal of Artificial Intelligence Research* 20 (2003), pp. 61–124. DOI: 10.1613/jair.1129.

[2] M. Hu, Y. Mu, X. Yu, et al. "Tree-Planner: Efficient Close-Loop Task Planning with Large Language Models". In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. 2024. URL: https://openreview.net/forum?id=Glcsog6zOe.

[3] OpenAI, J. Achiam, S. Adler, et al. "GPT-4 Technical Report". In: *arXiv* (2024). arXiv: 2303.08774.

[4] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. "Large Language Models Are Zero-Shot Reasoners". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 22199–22213.

[5] S. Kambhampati, K. Valmeekam, L. Guan, et al. "LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks". In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 22895–22907. URL: https://proceedings.mlr.press/v235/kambhampati24a.html.

[6] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati. "PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023, pp. 38975–38987.

[7] M. Aghzal, E. Plaku, G. J. Stein, and Z. Yao. "A Survey on Large Language Models for Automated Planning". In: *arXiv* (2025). arXiv: 2502.12435.

[8] X. Huang, W. Liu, X. Chen, et al. "Understanding the Planning of LLM Agents: A Survey". In: *arXiv* (2024). arXiv: 2402.02716.

[9] D. Gosmar and D. A. Dahl. "Hallucination Mitigation Using Agentic AI Natural Language-Based Frameworks". In: *arXiv* (2025). arXiv: 2501.13946.

[10] C. Rivera, G. Byrd, W. Paul, et al. "ConceptAgent: LLM-Driven Precondition Grounding and Tree Search for Robust Task Planning and Execution". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025.

[11] S. Yao, J. Zhao, D. Yu, et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. 2023. URL: https://openreview.net/forum?id=WE_vluYUL-X.

[12] S. Yao, D. Yu, J. Zhao, et al. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. URL: https://openreview.net/forum?id=5Xc1ecxO1h.

[13] C. Zhang, Z. Li, and W. Yuan. "Vote-Tree-Planner: Optimizing Execution Order in LLM-Based Task Planning Pipeline via Voting". In: *arXiv* (2025). arXiv: 2502.09749.

[14] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg. "Inference-Time Intervention: Eliciting Truthful Answers from a Language Model". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. DOI: 10.5555/3666122.3667919.

[15] R. E. Fikes and N. J. Nilsson. "STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving". In: *Artificial Intelligence* 2.3–4 (1971), pp. 189–208. DOI: 10.1016/0004-3702(71)90010-5.

[16] M. Ahn, A. Brohan, N. Brown, et al. "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances". In: *Proceedings of the 6th Conference on Robot Learning (CoRL)*. Vol. 205. Proceedings of Machine Learning Research. PMLR, 2022, pp. 287–318. URL: https://proceedings.mlr.press/v205/ichter23a.html.

[17] I. Singh, V. Blukis, A. Mousavian, et al. "ProgPrompt: Generating Situated Robot Task Plans Using Large Language Models". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 11523–11530. DOI: 10.1109/ICRA48891.2023.10161317.

[18] S. Shin, S. Kim, Y. Jang, M. Lee, and H. Woo. "Semantic Skill Grounding for Embodied Instruction-Following in Cross-Domain Environments". In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024, pp. 3354–3376. DOI: 10.18653/v1/2024.findings-acl.200.

[19] S. Chen, A. Xiao, and D. Hsu. "LLM-State: Open World State Representation for Long-Horizon Task Planning with Large Language Model". In: *arXiv* (2024). arXiv: 2311.17406.

[20] J. Wei, X. Wang, D. Schuurmans, et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837.

[21] S. Hao, Y. Gu, H. Ma, et al. "Reasoning with Language Model Is Planning with World Model". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023, pp. 8154–8173. DOI: 10.18653/v1/2023.emnlp-main.507.

[22] Y. Zhuang, X. Chen, T. Yu, et al. "ToolChain*: Efficient Action Space Navigation in Large Language Models with A* Search". In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. 2024. URL: https://openreview.net/forum?id=B6pQxqUcT8.

[23] K. Stein, D. Fišer, J. Hoffmann, and A. Koller. "Automating the Generation of Prompts for LLM-Based Action Choice in PDDL Planning". In: *Proceedings of the 35th International Conference on Automated Planning and Scheduling (ICAPS)*. Vol. 35. 1. 2025, pp. 250–259. DOI: 10.1609/icaps.v35i1.36126.

[24] T. Silver, S. Dan, K. Srinivas, J. B. Tenenbaum, L. P. Kaelbling, and M. Katz. "Generalized Planning in PDDL Domains with Pretrained Large Language Models". In: *Proceedings of the 38th AAAI Conference on Artificial Intelligence*. Vol. 38. 18. 2024, pp. 20256–20264. DOI: 10.1609/aaai.v38i18.30006.

[25] E. J. Hu, Y. Shen, P. Wallis, et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *Proceedings of the 10th International Conference on Learning Representations (ICLR)*. 2022. URL: https://openreview.net/forum?id=nZeVKeeFYf9.

[26] F. Meng, Z. Wang, and M. Zhang. "PiSSA: Principal Singular Values and Singular Vectors Adaptation of Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024. URL: https://openreview.net/forum?id=6ZBHIEtdP4.

[27] M. Helmert. "The Fast Downward Planning System". In: *Journal of Artificial Intelligence Research* 26 (2006), pp. 191–246. DOI: 10.1613/jair.1705.

[28] S. Basu, G. S. Ramachandran, N. S. Keskar, and L. R. Varshney. "Mirostat: A Neural Text Decoding Algorithm that Directly Controls Perplexity". In: *Proceedings of the 9th International Conference on Learning Representations (ICLR)*. 2021. URL: https://openreview.net/forum?id=W1G1JZEIy5_.

[29] M. Besta, N. Blach, A. Kubicek, et al. "Graph of Thoughts: Solving Elaborate Problems with Large Language Models". In: *Proceedings of the 38th AAAI Conference on Artificial Intelligence*. Vol. 38. 16. 2024, pp. 17682–17690. DOI: 10.1609/aaai.v38i16.29720.